UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE COMPUTACIÓN • SISTEMAS DISTRIBUIDOS

PONG // NEON

INFORME TÉCNICO DEL PROYECTO

Sockets UDP, programación multihilo, clases y principios SOLID



INTEGRANTES

Alejandro Ojeda · Christian Carrion

Trabajo desarrollado en conjunto

21 de octubre de 2026

Periodo: 69

Tabla de contenido

1. Introducción 3

2. Objetivos 3

Objetivo general 3

Objetivos específicos 3

3. Herramientas utilizadas 3

4. Funcionamiento general 4

5. Arquitectura cliente-servidor 4

5.1. Comunicación y multihilo 4

6. Clases, objetos y constructores 4

7. Principios SOLID aplicados 5

8. Cambios realizados respecto a la versión anterior 5

9. Reglas del juego 6

10. Ejecución básica 6

11. Pruebas y resultados 6

12. Dificultades y soluciones 7

13. Conclusiones 7

14. Trabajo en equipo y entregables 7



1. Introducción

En este proyecto desarrollamos Pong Neon, un videojuego de dos participantes creado con Python y Pygame. A diferencia de un Pong local, la partida se puede compartir entre computadoras mediante sockets UDP: un equipo ejecuta el servidor y dos clientes reciben el estado del juego y envían sus controles.

Inicialmente trabajamos con un código donde el movimiento, las colisiones y la pantalla estaban concentrados en el ciclo principal. En la actualización organizamos el programa con clases y objetos para hacerlo más claro, fácil de mantener y sencillo de explicar. Además, aplicamos principios SOLID sin perder la jugabilidad ni el diseño neón.

2. Objetivos

Objetivo general

Desarrollar y organizar un Pong multijugador usando Python, Pygame, sockets UDP, multihilo y programación orientada a objetos, aplicando principios SOLID a los principales componentes.

Objetivos específicos

- Mantener una partida de dos jugadores con puntuación y rebotes sincronizados por red.
- Separar el programa en clases con atributos, métodos y constructores.
- Aplicar los principios SOLID en el manejo del juego, el transporte de datos y la vista.
- Verificar rebotes, controles, puntuación, reinicio y envío de mensajes con pruebas automatizadas.

3. Herramientas utilizadas

Tecnología	Uso en el proyecto
Python 3.10 o superior	Lenguaje de programación y lógica del juego.
Pygame Community Edition	Ventana, raquetas, pelota, marcador y controles.
socket + UDP + JSON	Intercambio de controles y estados entre computadoras.
threading + Lock	Recepción simultánea de mensajes y acceso seguro al estado compartido.
unittest	Pruebas automatizadas de física, reglas y contratos de clases.
PyCharm / Ubuntu / Windows	Entornos de desarrollo y ejecución de los clientes.

4. Funcionamiento general

El servidor actúa como autoridad del juego: calcula la posición de la pelota, detecta colisiones, actualiza los puntos y determina quién gana. Cada cliente dibuja la misma partida con Pygame y solo envía las teclas que presiona su jugador. Esto evita que ambas computadoras calculen resultados diferentes.

5. Arquitectura cliente-servidor



Figura 1. Flujo general de datos entre el servidor y los dos clientes por UDP. Fuente: elaboración para este proyecto.

5.1. Comunicación y multihilo

La comunicación se realiza con sockets UDP y mensajes JSON. Los clientes informan entradas como subir o bajar; el servidor responde con el estado de pelota, raquetas, puntos, pausa y ganador. En el servidor, un hilo atiende los mensajes entrantes y el hilo principal actualiza la partida. En cada cliente, otro hilo recibe los estados mientras la ventana continúa dibujándose.

Para evitar conflictos al leer o modificar datos desde distintos hilos se utiliza threading.Lock(). También se emplean números de secuencia y reenvío breve de órdenes importantes, ya que UDP no garantiza que todos los paquetes lleguen ni que lleguen en el mismo orden.

6. Clases, objetos y constructores

Cada clase representa una parte concreta del sistema. Con __init__() asignamos los valores iniciales; con los métodos definimos qué puede hacer cada objeto. Por ejemplo, las dos raquetas son instancias independientes de Raqueta y cada una guarda su posición.

Capa / clases	Responsabilidad principal
Modelo: Pelota, Raqueta, Jugador	Movimiento, colisiones, controles y datos del participante.
Reglas: JuegoPong, ConfiguracionJuego	Puntuación, ganador, partida y parámetros configurables.
Red: Transporte, TransporteUDP, ConexionUDP	Contrato de transporte y mensajes de servidor/cliente.
Pantalla: VistaPongNeon, EfectosVisuales	Dibujar interfaz, brillo, estela y partículas.
Entrada y coordinación: ControladorEntrada, ServidorPong, ClientePong	Traducir teclas y ejecutar los ciclos de cada aplicación.

Ejemplo: JuegoPong(config) construye la partida y crea la Pelota y las dos Raquetas. Por separado, ServidorPong(juego, transporte) recibe por constructor el motor del juego y el objeto que realiza la comunicación. Esta separación hace que las piezas puedan probarse sin abrir la ventana gráfica.

7. Principios SOLID aplicados

Principio	Aplicación en Pong Neon
S · Responsabilidad única	Pelota maneja la física; Raqueta sus límites; VistaPongNeon dibuja; ConexionUDP recibe y envía datos.
O · Abierto/cerrado	Podemos crear otra implementación del contrato Transporte sin reescribir el coordinador ServidorPong.
L · Sustitución de Liskov	TransporteUDP implementa Transporte; la prueba usa un TransporteFalso compatible con ese contrato.
I · Segregación de interfaces	Transporte define solamente recibir(), enviar() y cerrar(), operaciones necesarias para la red.
D · Inversión de dependencias	ServidorPong recibe un objeto Transporte en su constructor y no crea directamente el socket UDP.

8. Cambios realizados respecto a la versión anterior

- Se pasó de bloques de lógica general a clases que representan pelota, raquetas, participantes y partida.
- Se separó el cálculo del servidor de la vista Pygame de los clientes.

- Se creó un contrato de transporte que reduce la dependencia del servidor respecto al socket concreto.
- Se mantuvieron UDP, recepción multihilo, marcador de dos jugadores y los controles W/S, flechas, P, C y R.
- Se conservó la estética neón, los efectos de la pelota y la barra inferior separada de la cancha.
- Se agregaron pruebas unitarias y una prueba local de comunicación entre dos clientes UDP.

9. Reglas del juego

El Jugador 1 controla su raqueta con W/S y el Jugador 2 con las flechas arriba/abajo. La pelota rebota en techo, suelo y raquetas; el ángulo cambia según el lugar donde golpea. Cada choque con una raqueta aumenta la rapidez un 5 % hasta 900 px/s. Si la pelota supera una raqueta, el rival anota un punto. Gana quien llegue primero a siete puntos. P pausa, C continúa, R reinicia y ESC cierra el cliente.

10. Ejecución básica

En Windows, el servidor se ejecuta con: python servidor.py --bind 0.0.0.0 --puerto 5000. El Jugador 1 usa: python cliente.py --host 127.0.0.1 --puerto 5000. En la segunda computadora (Windows o Ubuntu), el Jugador 2 usa: python cliente.py --host IP_ETHERNET_DEL_SERVIDOR --puerto 5000. La dirección Ethernet actual se obtiene con ipconfig y puede variar. Cada cliente debe instalar pygame-ce.

11. Pruebas y resultados

Para revisar el refactor, ejecutamos pruebas de lógica y una prueba de red local. Los siguientes resultados corresponden al código actual de las clases y fueron obtenidos el 21 de octubre de 2026.

Comprobación	Resultado
Límites de las raquetas	Aprobada: las raquetas permanecen dentro de la ventana.
Rebote en pared superior	Aprobada: la pelota invierte correctamente su velocidad vertical.
Aceleración progresiva	Aprobada: incremento del 5 % al tocar una raqueta.
Puntuación y reinicio	Aprobada: ganador a siete puntos y retorno del marcador a cero.
Comandos y secuencias UDP	Aprobada: pausa, continuación, reinicio y rechazo de secuencias antiguas.
Intercambio de transporte	Aprobada: el servidor acepta un transporte de prueba.

Prueba UDP local con dos clientes	Aprobada: registro, movimiento, P/C/R y sincronización.

Resultado total: 6/6 pruebas unitarias aprobadas y prueba UDP local aprobada. Estas verificaciones no sustituyen una prueba manual completa de interfaz ni una ejecución cruzada Windows-Ubuntu de la versión refactorizada; esa validación debe repetirse al desplegar el proyecto en la red universitaria.

12. Dificultades y soluciones

- Conectividad: al principio se intentó utilizar una IPv4 Wi-Fi que cambió; se solucionó comprobando ipconfig y utilizando la dirección Ethernet activa del servidor.
- Pantalla: la barra de controles tapaba la pelota cerca del borde inferior; se separó la zona visual de juego de la barra informativa.
- Organización: al crecer el código se mezclaban la red, las reglas y el dibujo; se reorganizó el proyecto usando POO y responsabilidades separadas.

13. Conclusiones

Con este trabajo comprendimos mejor cómo se relacionan un servidor y dos clientes en una aplicación distribuida. UDP permitió intercambiar datos del juego y los hilos hicieron posible escuchar la red sin detener la actualización de la partida.

La actualización a clases y objetos facilitó identificar el trabajo de cada componente. También comprobamos que SOLID es útil cuando queremos corregir una parte del programa sin afectar a todas las demás. El proyecto mantiene su apariencia y funcionalidad, pero ahora está mejor estructurado para continuar agregando características.

14. Trabajo en equipo y entregables

Integrantes: Alejandro Ojeda y Christian Carrion.

El trabajo se realizó en conjunto y se entrega con servidor.py, cliente.py, README.md, requirements.txt y tests/. Como referencias técnicas se utilizaron el propio código y las documentaciones de Python y Pygame CE.